

Autor: Zărnescu Raul-Claudiu

Data: 04.07.2026

Versiunea: v7

Modificări v1 → v2:

- Corectarea numerotării capitolelor și reordonarea Motorului AI înaintea Implementării Software.
- Fast-Path: înlocuirea whitelisting-ului static pe clase de IP (CDN-uri) cu Dynamic Flow Offloading.
- Slow-Path: adăugarea fallback-urilor pentru ECH și DoH (Canary Domains).
- Honeypot Routing: handoff către Netfilter/conntrack în loc de rescriere manuală de headere în eBPF.
- Documentarea autentificării Control Plane API (token + mTLS opțional + rate limiting).
- Adăugarea 7.3 — Single Point of Failure (Fail-Open software + Hardware Watchdog).
- Adăugarea Cap. 8 — Fiabilitate Hardware (SSD + Read-Only RootFS + tmpfs).
- Adăugarea Cap. 9 — Resurse Hardware (BOM).
- Adăugarea scenariului de test 10.5 — Failover/Recovery.

Modificări v2 → v3:

- Separarea Out-of-Band în VLAN 99 (management) și VLAN 100 ("Dirtnet", Honeypot).
- Definirea ordinii de evaluare a regulilor eBPF (3.1.1): Rate Limiting → Blocklist → Honeypot → Dynamic Flow Offload → Slow-Path.
- Politică de evacuare LRU/TTL pentru eBPF Map-ul de Dynamic Flow Offloading.

Modificări v3 → v4:

- Rezolvarea conflictului Layer 2/Layer 3 pe Bridge: br_netfilter + subinterfață VLAN dedicată pentru expunerea API-ului.
- Optimizarea eBPF: eliminarea Garbage Collector-ului din user-space (evacuare strict LRU, nativă în kernel).
- Delegarea parsării DNS către Slow-Path (limitări eBPF Verifier) și eliminarea race condition-ului la TCP Tarpitting.
- Mitigare Slowloris: Concurrent Connections Limit direct în XDP.

Modificări v4 → v5:

- Clarificarea configurării VLAN 99/100 pe eth1: subinterfețe 802.1Q dedicate, excluse din br0.
- Tratarea explicită a traficului IPv6 ca "necunoscut" în pipeline (secțiune nouă 3.4).
- Corectarea pierderii vizibilității IP-ului atacatorului prin SNAT/Masquerade: jurnal de corelare DNAT/SNAT + câmpuri source_ip/timestamp în XAI.
- Mitigare TOCTOU în Dynamic Flow Offloading: TTL de re-validare periodică.
- Contor agregat (global) de Rate Limiting, complementar celui per-sursă.
- Izolare explicită Honeypot/Motor AI pe stația secundară (network namespace/VM dedicat).
- Strategie de actualizare OTA (A/B) compatibilă cu rootfs read-only (8.3).
- Politică minimă de backup pentru SSD-ul extern (8.4).

- Documentarea bypass-ului hardware (Network TAP/relevu Fail-Open electric) ca limitare cunoscută dacă nu e achiziționat.
- Documentarea absenței UPS ca limitare cunoscută, neimplementată.

Modificări v5 → v6:

- Eliminarea SNAT/Masquerade pentru Honeypot (2.2, 4.2, 5.1): VLAN 100 fiind un segment izolat cu Raspberry Pi ca unic gateway posibil, rutarea de retur e deja simetrică prin construcție — SNAT era o complexitate auto-indusă, care rezolva o problemă creată chiar de alegerea SNAT. Un DNAT simplu păstrează nativ IP-ul real al atacatorului direct în logurile Honeypot-ului, eliminând totodată necesitatea jurnalului de corelare DNAT/SNAT introdus în v5 (și, implicit, nevoia unui endpoint API dedicat pentru interogarea lui).
- Optimizare de performanță pentru br_netfilter (3.1.1, 5.1, 10.4): introducerea regulii NOTRACK pentru traficul deja promovat prin Dynamic Flow Offloading, astfel încât activarea globală a br_netfilter (necesară pentru Honeypot) să nu impună overhead de conntrack pe tot traficul de producție.
- Adăugarea unei politici de evacuare (TTL inline) pentru contorul de conexiuni concurente per-sursă din 5.4, simetrică cu politica deja aplicată hărții de Dynamic Flow Offloading, pentru a preveni "scurgeri" ale contorului care ar bloca permanent surse legitime.
- Clarificarea explicită a mecanismului de re-validare TTL din 3.1: verificarea se face inline (comparație de timestamp la fiecare lookup, cost $O(1)$), nu printr-o scanare periodică din user-space — consecvent cu decizia din v4 de a elimina Garbage Collector-ul extern.
- Recalibrarea KPI-urilor din 10.4 pentru a reflecta explicit costul rezidual al br_netfilter/conntrack după optimizarea NOTRACK.

Modificări v6 → v7:

- Notă nouă în 3.1.1: sincronizarea ciclului de viață dintre TTL-ul eBPF al fluxurilor promovate și regula NOTRACK asociată din nftables — dacă intrarea din eBPF Map expiră fără ca regula NOTRACK să expire/fie ștearsă simultan, pachetul retrogradat spre Slow-Path riscă să nu mai fie urmărit corect de conntrack.
- Notă nouă în 3.4: riscul de saturare CPU pe Raspberry Pi dacă volumul de trafic IPv6 nativ (ex: streaming) e mare, dat fiind că Scapy (Slow-Path) e semnificativ mai lent decât eBPF; documentată explicit opțiunea de contingentă (DROP complet pe IPv6, forțând fallback pe IPv4) dacă Slow-Path nu face față volumului.

Sumar al proiectului

Proiectul adoptă o arhitectură hibridă, structurată pe două paliere, pentru a asigura un echilibru optim între viteza de procesare a rețelei și performanța analizei de securitate.

Primul palier constă într-un dispozitiv IPS (Intrusion Prevention System) integrat "in-line" între ruterul principal și consumatori. Implementarea se bazează pe următoarea topologie de rețea: **Ruter 1 (Gateway) → Raspberry Pi (IPS Transparent Bridge) → Ruter 2 (Mod Access Point / Switch)**. Această topologie creează un sub-domeniu (subnet) complet monitorizat, acționând la nivelul Layer 2. Astfel, eliminăm necesitatea configurării unor tabele de rutare complexe (și riscul de Double NAT) pe dispozitivul Raspberry Pi.

Pe platforma IPS va rula o distribuție de Linux Server. Obiectivul principal este integrarea logicii de decizie la nivel de kernel, utilizând un program **eBPF/XDP** atașat direct la driverul interfeței de rețea. Ca metodă de fallback, se va dezvolta o aplicație în user-space utilizând **Netfilter Queue** (NFQUEUE).

Deși NFQUEUE este mai simplu de implementat, scriptul direct în kernel (eBPF) este prioritar pentru a evita latența introdusă de "context switching".

Motorul de inspecție IPS va folosi o arhitectură de tip **Fast-Path / Slow-Path**:

- **Fast-Path**: Funcționează ca un filtru de microsecundă la Layer 3 (Adrese IP) și Layer 4 (Porturi TCP/UDP). În locul unei liste statice de surse de încredere, sistemul implementează **Dynamic Flow Offloading**: un flux este promovat la trecere directă doar după ce a fost validat repetat de Slow-Path (vezi 3.1).
- **Slow-Path**: Reprezintă inspecția avansată la Layer 7. Evaluarea se bazează pe **Handshake Analysis** (SNI, JA3/JA4) și **Encrypted Traffic Analytics (ETA)**, cu fallback dedicat pentru trafic ECH și DoH (vezi 3.2), și tratare explicită a traficului IPv6 ca "necunoscut" (vezi 3.4).

Al doilea palier este **Sistemul Out-of-Band (AI)**. Acesta va asimila traficul din zona "gri" — conexiuni direcționate către Slow-Path care prezintă anomalii comportamentale, dar pentru care IPS-ul nu deține o semnătură de blocare definitivă. IPS-ul va replica și transmite exclusiv metadatele acestor conexiuni către motorul AI, prin canalul de management izolat (VLAN 99).

Totodată, sistemul face tranziția de la protecție pasivă la *Active Defense*. Orice amenințare confirmată nu va fi doar blocată, ci traficul atacatorului va fi redirectionat printr-un handoff controlat către stiva standard de kernel (Netfilter/conntrack), într-un mediu izolat de tip **"Honeypot"** (pe VLAN 100) pentru a i se profila comportamentul (vezi 5.1).

2. Topologia Rețelei și Arhitectura de Implementare

Proiectul utilizează o topologie segmentată care separă traficul de date al consumatorilor (Data Plane) de traficul de securitate și telemetrie (Management/Control Plane), evitând fenomenul de "Double NAT" și minimizând latența.

2.1. Arhitectura Fizică (The Data Plane)

Dispozitivul Raspberry Pi acționează ca un senzor și actuator in-line.

- **Nodul 1 (Perimetru)**: Ruterul principal (Gateway), responsabil de conexiunea WAN, NAT și alocarea DHCP.
- **Nodul 2 (IPS In-line)**: Raspberry Pi configurat ca un "Transparent Layer 2 Bridge" (br0). Interfața nativă gigabit (eth0) preia traficul de la Gateway, iar un adaptor USB 3.0-to-Gigabit Ethernet (eth1) trimite traficul mai departe.
- **Nodul 3 (Distribuție)**: Un al doilea ruter configurat strict în modul Switch/Access Point, care distribuie traficul curat către rețeaua locală.

Notă (Limitare Hardware / vezi Cap. 9 — Resurse Hardware): Adaptorul USB 3.0-to-Gigabit Ethernet (eth1) este oferit de mentorul de practică; specificațiile exacte ale chipset-ului nu sunt cunoscute la momentul redactării. Majoritatea adaptoarelor USB Ethernet NU suportă XDP nativ (driver mode), ci doar XDP generic (SKB mode), semnificativ mai lent. Latența de "microsecundă" promisă pentru Fast-Path este garantată doar pe interfața eth0 (nativă); performanța reală pe eth1 va fi validată experimental imediat ce piesa este disponibilă.

Notă tehnică (Expunerea Control Plane-ului pe Bridge): Deși Raspberry Pi-ul operează ca un bridge transparent la Layer 2 (fără rutare pentru traficul de tranzit), expunerea API-ului de Control Plane către rețeaua de management (VLAN 99) necesită conectivitate la Layer 3. Pentru a rezolva acest conflict, se va instanția o subinterfață 802.1Q dedicată (ex: eth1.99), montată direct peste

interfața fizică eth1 și explicit exclusă din br0 — nu peste bridge — tocmai pentru a evita ca interfața de management să vadă traficul de tranzit al consumatorilor (sau invers). Acestei subinterfețe i se va alocă un IP static local. Această interfață va asculta strict request-urile mTLS/autentificate destinate platformei IPS, fără a influența traficul de tranzit al consumatorilor.

2.2. Conexiunea Out-of-Band (The Control Plane)

Pentru a comunica cu motorul de inteligență artificială și pentru a găzdui mediul Honeypot (pe o stație de calcul secundară), sistemul utilizează o arhitectură Out-of-Band (OoB), împărțită în **două canale logice complet separate**, pentru a preveni contaminarea rețelei de management cu traficul atacatorului:

- **VLAN 99 — Management / Telemetrie (canal de încredere):** transportă exclusiv trafic care NU provine de la un atacator — comenzile Control Plane API (block/tarbit/honeypot, vezi 6.3), stream-ul de metadate extras de Slow-Path (SNI, JA3/JA4) și alertele modelului ML. Acest VLAN este complet invizibil pentru orice trafic redirectionat către Honeypot.
- **VLAN 100 — "Dirtnet" (canal izolat pentru atacator):** transportă exclusiv traficul unui atacator redirectionat către mediul Honeypot (handoff descris în 5.1). Este segmentul pe care atacatorul îl poate, teoretic, vedea sau ataca dintr-un container escape sau dintr-o scanare de rețea lansată din interiorul Honeypot-ului — de aceea NU are conectivitate către VLAN 99, către API-ul de control sau către restul infrastructurii de management.

Notă (Risc Corectat — Contaminarea Planului de Management): în versiunea inițială a proiectului, telemetria/API-ul și traficul redirectionat al atacatorului foloseau același VLAN. Separarea strictă în VLAN 99 (management, invizibil atacatorului) și VLAN 100 ("dirtnet", strict pentru Honeypot) elimină acest risc.

Sistemul va fi implementat folosind una dintre următoarele abordări de izolare logică/fizică, aplicate distinct pentru cele două VLAN-uri:

- **Metoda de Separare Logică (VLAN 802.1Q, dublă):** Utilizarea conexiunii fizice existente (eth1 către switch-ul intern), configurată ca **trunk pentru două VLAN-uri tagged**: VLAN 99 (management) și VLAN 100 (dirtnet). Traficul valid al consumatorilor rămâne *untagged*, pe rețeaua de producție.
- **Metoda de Separare Fizică (Dedicated Management Interface):** Utilizarea interfeței Wi-Fi native (wlan0) sau a unui al treilea port Ethernet (eth2) exclusiv pentru VLAN 99, complet separată fizic de VLAN 100 (dirtnet), care rămâne pe eth1 (tagged).

Notă (Risc de Compatibilitate): Metoda de Separare Logică presupune ca Ruterul 2 (configurat ca Switch/AP) să suporte trunking 802.1Q pentru ambele VLAN-uri simultan. Dacă routerul nu suportă trunking dual, proiectul adoptă implicit Metoda de Separare Fizică descrisă mai sus.

Notă de Configurare (v6 — Rutare Simetrică Nativă, fără SNAT): Gateway-ul implicit al stației Honeypot pe VLAN 100 trebuie configurat explicit ca fiind subinterfața Raspberry Pi pe acest VLAN (ex: eth1.100), nu switch-ul sau alt dispozitiv din rețea. Deoarece VLAN 100 este, prin definiție, un segment complet izolat fără altă cale de ieșire, această configurare garantează că traficul de retur al Honeypot-ului nu are altă rută disponibilă decât înapoi prin Pi — rutarea este deci simetrică prin construcție. Acest lucru elimină necesitatea oricărei translații de adresă sursă (SNAT/Masquerade) pentru traficul redirectionat către Honeypot (vezi justificarea completă în 5.1); varianta inițială cu SNAT rezolva, de fapt, o problemă de rutare asimetrică pe care alegerea SNAT însăși o introducea, nu una inherentă arhitecturii.

3. Arhitectura de Inspecție Dual-Tier (Fast-Path / Slow-Path)

Notă (Izolare pe Stația Secundară — dincolo de segmentarea VLAN): separarea la nivel de VLAN (99/100) protejează rețeaua, dar nu înlocuiește izolarea locală pe stația AI + Honeypot. Mediul Honeypot (Cowrie) rulează într-un mediu izolat dedicat (VM sau container cu network namespace propriu, legat exclusiv de subinterfața VLAN 100), fără acces la stiva de rețea sau la procesele motorului AI, care rulează separat, legat exclusiv de subinterfața VLAN 99. Astfel, un container escape din Honeypot rămâne limitat la namespace-ul/VM-ul izolat, chiar dacă ambele componente coexistă pe același hardware fizic.

Pentru a menține viteza la nivel de linie fără a compromite securitatea, sistemul implementează o arhitectură de procesare asimetrică: **Fast-Path** și **Slow-Path**.

3.1. Nivelul Fast-Path (Inspecție la Layer 3 / Layer 4)

Fast-Path-ul este prima linie de decizie și operează exclusiv în spațiul kernel-ului (Kernel Space), utilizând hook-uri **eBPF/XDP** atașate direct la driverul plăcii de rețea.

- **Mecanism de funcționare:** La sosirea unui pachet, programul eBPF parsează exclusiv headerurile de nivel rețea și transport (Adrese IP Sursă/Destinație, Porturi TCP/UDP și Flag-uri).
- **Dynamic Flow Offloading:** Motorul eBPF menține o **eBPF Map de tip "flow cache"** populată dinamic. Un flux (5-tuple) este promovat la **XDP_PASS** necondiționat abia după ce a fost inspectat de un număr minim de pachete în Slow-Path și clasificat repetat drept benign. Această abordare elimină riscul ca un atacator să găzduiască infrastructură malițioasă în spatele unui IP aparținând unui furnizor de încredere (*domain/CDN fronting*), pentru că încrederea se câștigă per-flux, pe bază de comportament observat.
- **Politica de Evacuare a eBPF Map (Strict LRU):** harta de "flow cache" este declarată obligatoriu de tip **BPF_MAP_TYPE_LRU_HASH**, care evacuează automat și nativ, la nivel de kernel, intrările cel mai puțin recent folosite. Sistemul renunță intenționat la un Garbage Collector din user-space (Python): scanarea constantă a unei hărți eBPF active din spațiul utilizator ar introduce lock contention, blocând procesarea la nivel de XDP. Managementul stării rămâne exclusiv în sarcina nucleului Linux.
- **Sincronizarea cu Netfilter Contrack:** promovarea dinamică a unui flux de la Slow-Path înapoi pe Fast-Path (XDP_PASS necondiționat) nu desincronizează stiva kernel-ului. Odată promovat, modulul conntrack reține starea existentă a conexiunii (până la timeout-ul TCP implicit), asigurând că lipsa vizibilității pachetelor subsecvente în stiva superioară nu va genera pachete RST native din partea Pi-ului.
- **Re-validare Periodică (Mitigare TOCTOU):** promovarea la XDP_PASS necondiționat nu este permanentă. Fiecare intrare din eBPF Map de flow cache poartă, pe lângă contorul LRU, un TTL de re-validare (ex: 2-5 minute). La expirare, fluxul este retrogradat automat spre Slow-Path pentru o nouă rundă de inspecție. Această măsură limitează fereastra de expunere în care un atacator ar putea folosi un flux deja validat pentru abuz nedetectat (vezi 7.1).

*Precizare de implementare (v6): verificarea TTL-ului de re-validare se face **inline**, în programul eBPF, printr-o simplă comparație de timestamp la momentul lookup-ului în hartă (cost $O(1)$ per pachet, folosind `bpf_ktime_get_ns()`) — NU printr-o scanare periodică a hărții din user-space. Această precizare este necesară pentru a rămâne consecvenți cu decizia de mai sus de a elimina Garbage Collector-ul extern tocmai din motive de lock contention: un TTL verificat inline, la fiecare hit, nu reintroduce problema de performanță pe care eliminarea GC-ului o rezolva.*

- **Blocarea atacurilor volumetrice:** Pachetele care provin din surse confirmate ca fiind malițioase sau pachetele malformate primesc verdictul **XDP_DROP** înainte ca sistemul de operare să aloce memorie pentru ele.
- **Tranziția:** Traficul necunoscut, care nu poate fi nici promovat prin Dynamic Flow Offloading, nici blocat, este marcat și avansat către Slow-Path.

3.1.1. Ordinea de Evaluare a Regulilor (Pipeline eBPF)

Pipeline-ul este definit explicit, în această ordine strictă, evaluată pentru fiecare pachet:

- **1. Rate Limiting (Anti-DDoS) — necondiționat:** contorul PPS per sursă/flux este verificat PRIMUL, pentru absolut tot traficul, indiferent de orice altă clasificare anterioară. Dacă pragul e depășit → **XDP_DROP** imediat (vezi 5.4).
- **2. Blocklist static:** dacă sursa se află pe lista neagră confirmată → **XDP_DROP**.
- **3. Marcaj Honeypot / Decepție:** dacă fluxul e marcat explicit pentru redirectionare → **XDP_PASS** (handoff către Netfilter/conntrack, vezi 5.1).
- **4. Dynamic Flow Offload:** dacă fluxul se regăsește în eBPF Map ca fiind deja validat → **XDP_PASS**.
- **5. Implicit — Slow-Path:** orice pachet care nu se încadrează în pașii 1-4 este marcat și avansat către Slow-Path.

Notă: ordinea de mai sus rezolvă explicit scenariul în care o sursă deja direcționată către Honeypot lansează un atac volumetric — pasul 1 intervine înaintea pasului 3, oprind acest trafic prin XDP_DROP înainte să ajungă la Netfilter.

Optimizare de performanță (v6 — NOTRACK pentru trafic promovat): activarea globală a `br_netfilter` (necesară pentru handoff-ul către Honeypot, vezi 5.1) face ca, implicit, TOT traficul care traversează `br0` — inclusiv fluxurile deja promovate prin Dynamic Flow Offloading (pasul 4) — să fie evaluat de tabelele `iptables/nftables` și ținut în `conntrack`. Acest lucru contrazice parțial premisa de "viteză de linie fără costuri suplimentare" a Dynamic Flow Offloading. Mitigare: pentru fluxurile promovate la pasul 4 (deja validate, fără nevoie de DNAT către Honeypot), se instanțiază o regulă `iptables -t raw -j NOTRACK`` (sau echivalentul `nftables`), care exceptează explicit acest trafic de la urmărirea de stare în `conntrack`. Doar traficul marcat la pasul 3 (Honeypot) continuă să treacă prin `conntrack` complet, pentru a beneficia de DNAT/un-DNAT. Costul de performanță al `br_netfilter` rămâne astfel limitat la traficul care chiar are nevoie de el. Vezi și recalibrarea KPI-urilor din 10.4.

Atenție la implementare (v7 — Sincronizarea Ciclului de Viață TTL eBPF ↔ NOTRACK): TTL-ul de re-validare al fluxului promovat (3.1) trăiește exclusiv în eBPF Map-ul din kernel, în timp ce regula NOTRACK trăiește ca intrare separată în `nftables`. Cele două nu expiră automat împreună. Dacă intrarea din eBPF Map expiră și fluxul e retrogradat spre Slow-Path, dar regula NOTRACK corespunzătoare rămâne activă (sau invers — regula NOTRACK expiră/e ștersă, dar harta eBPF încă marchează fluxul ca promovat), pachetul retrogradat poate ajunge în Slow-Path fără ca Netfilter să îl mai urmărească corect în `conntrack`, sau poate rămâne needitat de `conntrack` când ar trebui. Soluția pe teren, cel mai probabil: gestionarea regulii NOTRACK prin API-ul ``nftables`` din Python (nu prin apeluri statice ``iptables``), cu un timeout setat identic cu TTL-ul din eBPF, și cu ștergerea explicită a regulii din user-space în același moment în care componenta care gestionează harta detectează/anticipează expirarea. Acest lucru trebuie tratat cu atenție la scrierea codului, ca cerință de sincronizare între cele două componente, nu ca detaliu implicit.

3.2. Nivelul Slow-Path (Inspecție Metadate la Layer 7)

Slow-Path-ul este rezervat exclusiv conexiunilor neclasificate. Arhitectura renunță la abordarea de tip *Full TLS Decryption (MITM)*, folosind în schimb **Encrypted Traffic Analytics (ETA)**.

- **Filtrarea SNI:** citirea domeniului cerut în clar din *ClientHello*; conexiuni către domenii C2 cunoscute sunt terminate instant (TCP Reset). Presupune SNI necriptat — vezi fallback ECH mai jos.
- **Amprentarea Criptografică (JA3/JA4):** hash-ul parametrilor de negociere TLS e comparat cu o bază de date, permițând blocarea unui script malițios chiar dacă payload-ul e criptat.
- **Analiza Dinamicii Pachetelor:** secvențialitate, mărime și inter-arrival time, pentru detectarea exfiltrării sau a shell-urilor interactive mascate.
- **Fallback pentru Encrypted Client Hello (ECH):** la detectarea extensiei ECH, fluxul e marcat "SNI indisponibil", comutând exclusiv pe JA3/JA4 și ETA, cu pondere crescută pentru analiza comportamentală.
- **Fallback pentru DNS-over-HTTPS (DoH) — Canary Domains:** traficul HTTPS către endpoint-uri cunoscute de DoH public (Cloudflare, Google) e fie blocat direct, fie marcat cu prioritate ridicată pentru Slow-Path.

3.3. Sinergia cu Sistemul Out-of-Band

Traficul din Slow-Path cu anomalii, dar fără prag de certitudine suficient pentru blocare definitivă, este lăsat să treacă. Metadatele conexiunii sunt clonate și transmise asincron (prin **VLAN 99/Syslog**) către stația secundară **AI Out-of-Band** pentru analiză și profilare.

3.4. Tratarea Traficului IPv6 (Trafic Necunoscut)

Arhitectura din 3.1-3.3 a fost concepută și validată pentru IPv4. Traficul IPv6 nu este blocat necondiționat, ci tratat explicit ca trafic "necunoscut", cu nivel de inspecție redus dar nenul.

- **Fast-Path:** cadrele IPv6 (EtherType 0x86DD) sunt identificate, dar fără parsarea completă a lanțului de Extension Headers, pentru că *eBPF Verifier*-ul nu acceptă parcurgerea unui lanț de lungime variabilă fără o limită fixă. Programul eBPF aplică totuși un Rate Limiting generic pe pachete/secundă la nivelul adresei sursă IPv6.
- **Slow-Path:** pachetele IPv6 care depășesc pragul de Rate Limiting sunt avansate către Scapy, care parsează headerul și extension headers fără restricțiile Verifier-ului, aplicând aceleași mecanisme SNI/JA3/JA4/ETA. DNS Sinkholing este extins pentru interogări AAAA.
- **Limitare asumată:** până la un suport nativ complet în eBPF, traficul IPv6 nu beneficiază de Dynamic Flow Offloading, ci doar de protecție anti-DDoS generică și inspecție completă (dar mai lentă) din Slow-Path — compromis acceptabil pentru stadiul curent (vezi 7.2).

*Risc de saturare CPU (v7): Scapy (folosit în Slow-Path pentru parsarea IPv6, vezi mai sus) este semnificativ mai lent decât procesarea nativă eBPF. Dacă rețeaua are un volum mare de trafic IPv6 nativ (ex: streaming video pe IPv6, tot mai frecvent la Netflix/YouTube), CPU-ul Raspberry Pi-ului poate satura rapid doar din procesarea acestui trafic în Slow-Path — un risc real pentru un MVP pe hardware ARM cu resurse limitate. Contingență documentată explicit: dacă testele de teren arată că Slow-Path/Scapy nu face față volumului de IPv6, echipa poate opta pentru **XDP_DROP necondiționat pe tot traficul IPv6** la nivelul Fast-Path, forțând clienții din rețea să treacă pe IPv4 (majoritatea sistemelor de operare moderne fac automat acest fallback prin Happy*

Eyeballs). Aceasta ar fi o înăsprire deliberată a limitării deja asumate mai sus, nu o soluție de arhitectură nouă, și ar trebui aplicată doar dacă metricile de CPU din 10.4 arată saturare reală.

4. Motorul de Inteligență Artificială Out-of-Band și Logica de Analiză

Motorul AI reprezintă creierul analitic al arhitecturii de Active Defense. Sistemul utilizează exclusiv modele AI rulate local (On-Premise) pe stația secundară Out-of-Band, pentru confidențialitate și latență minimă.

4.1. Detecția Anomaliilor de Rețea (Machine Learning Tradițional)

- **Modelul ales:** Algoritmi de tip **Isolation Forest** sau **Random Forest**, antrenați periodic pe logurile de *baseline* ale rețelei.
- **Vectorii de extracție:** semnături JA3/JA4 necunoscute, *Inter-Arrival Time*, asimetria volumului de date.
- **Motivația:** algoritmi eficienți pe CPU și nativ explicabili (Feature Importance).

4.2. Profilarea Atacatorului și XAI (Local Large Language Model)

Sistemul integrează un model lingvistic de dimensiuni reduse (SLM/LLM), rulat local (ex: *Llama-3 8B* sau *Mistral-7B*), specializat pe securitate cibernetică prin una dintre:

- **Metoda A — Fine-Tuning (LoRA/QLoRA):** re-antrenare parțială pe loguri reale de terminal (ex: atacuri capturate în Cowrie), mapate pe rezultate structurate.
- **Metoda B — RAG (Retrieval-Augmented Generation):** interogarea unei baze de date vectoriale locale (MITRE ATT&CK, CVE) înainte de analiza logului, pentru a limita halucinațiile.

Generarea Alertelor XAI: LLM-ul formatează comenzile atacatorului și rezultatele modelului de Isolation Forest într-un JSON standardizat:

- **verdict:** "Malicious" / "Benign"
- **source_ip:** adresa IP reală a atacatorului. Datorită rutării simetrice native descrise în 2.2 și 5.1 (Honeypot-ul are ca gateway implicit subinterfața Pi pe VLAN 100, fără SNAT/Masquerade), Honeypot-ul vede și înregistrează direct, în propriile sale loguri, adresa IP reală a atacatorului — fără a fi necesară nicio corelare suplimentară pe Raspberry Pi. Motorul AI preia acest câmp direct din logurile Honeypot-ului (Cowrie).
- **timestamp:** momentul UTC al evenimentului, util pentru corelarea cu jurnalul de contrack și cu alte surse de log în cadrul unei investigații.
- **confidence_score:** 0.0 - 1.0
- **mitre_tactic_identified:** Ex: "TA0004 - Privilege Escalation"
- **action_taken:** "tarpit" / "honeypot" / "block"
- **human_readable_reasoning:** explicația sintetică a deciziei.

4.3. API-ul de Feedback și Închiderea Buclei (Closed-Loop Defense)

Odată ce motorul AI confirmă o amenințare cu un `confidence_score` peste prag (ex: > 0.85), declanșează un request HTTPS POST către Control Plane API-ul Raspberry Pi-ului, care injectează instantaneu instrucțiunea în structurile eBPF din kernel.

Notă: mecanismul de autentificare al acestui request (token / mTLS / rate limiting) este documentat integral în secțiunea 6.3.1.

5. Stratul de Decepție și Mecanismele de Active Defense

Blocarea imediată (DROP) informează atacatorul că a fost detectat. Pentru a extrage Threat Intelligence, acest proiect implementează *Active Defense* bazată pe decepție cibernetică.

5.1. Redirecționarea către Honeypot (Handoff Fast-Path → Netfilter/Conntrack)

Varianta inițială presupunea rescrierea directă a adreselor MAC/IP în eBPF. Arhitectura adoptată delegă gestiunea stării către stiva standard de kernel:

- **Rolul Fast-Path (XDP) rămâne strict cel din Cap. 3:** doar XDP_PASS (trafic legitim/promovat) și XDP_DROP (atacuri volumetrice). Fast-Path NU gestionează el însuși redirecționarea.
- **Marcarea atacatorului:** când sursa e confirmată ostilă, dar decizia e "redirecționare" (nu "blocare"), eBPF aplică XDP_PASS, lăsând pachetele să urce în stiva standard.
- **Handoff către Netfilter și Rezolvarea Paradoxului L2/L3:** deoarece Pi-ul e un bridge transparent la Layer 2, pachetele nu traversează implicit tabelele nat/conntrack de Layer 3. Se activează modulul `br_netfilter` și ``net.bridge.bridge-nf-call-iptables=1``, forțând pachetele de pe bridge să fie evaluate de lanțurile Layer 3. (Vezi 3.1.1 pentru optimizarea NOTRACK care limitează costul acestei activări globale la traficul care chiar are nevoie de conntrack.)
- **Rutare Simetrică Nativă (DNAT simplu, fără SNAT — corectat în v6):** Traficul preluat de iptables/nftables suferă doar o translație **DNAT**, care schimbă destinația către mediul Honeypot (VLAN 100). Nu este necesară nicio regulă de SNAT/Masquerade: deoarece VLAN 100 este un segment complet izolat (vezi 2.2), în care Raspberry Pi este singurul gateway posibil pentru stația Honeypot, traficul de retur al Honeypot-ului nu are altă rută disponibilă decât înapoi prin Pi. Rutarea este deci simetrică prin construcție, nu prin translație artificială de adresă. Modulul de conntrack al Pi-ului reface automat translația pe drumul de retur (un-DNAT), iar Honeypot-ul vede și înregistrează direct adresa IP reală a atacatorului, fără a fi nevoie de niciun jurnal de corelare suplimentar.

Notă (Corectare v6): versiunea anterioară introducea o regulă de SNAT/Masquerade "pentru a rezolva rutarea asimetrică", însoțită de un jurnal de corelare DNAT/SNAT pe Raspberry Pi pentru a recupera IP-ul real al atacatorului, pierdut din cauza SNAT. Analiza a arătat că problema era auto-indusă: VLAN 100 fiind un segment izolat cu Pi ca unic gateway, rutarea de retur e deja simetrică fără nicio translație de sursă. Eliminarea SNAT-ului elimină, în lanț, și nevoia jurnalului de corelare și a oricărui endpoint API suplimentar pentru interogarea lui — o simplificare arhitecturală, nu doar cosmetică.

- **Beneficiul abordării:** conntrack este un mecanism matur al kernel-ului Linux, care rezolvă corect o problemă pe care rescrierea manuală în eBPF ar fi gestionat-o dificil. Costul (o latență ușor mai mare) e acceptabil pentru că se aplică doar traficului deja marcat drept suspect.

- **Segmentare de rețea obligatorie (vezi 2.2):** traficul DNAT-at către Honeypot este întotdeauna direcționat pe **VLAN 100**, complet separat de VLAN 99. Chiar dacă atacatorul reușește un container escape, nu are conectivitate către API-ul de control sau stația AI.
- **Efectul final:** Atacatorul crede că a compromis un server legitim și rulează scripturi/comenzi. Sistemul OOB înregistrează în siguranță payload-ul și TTPs, furnizând AI-ului date brute pentru profilare.

5.2. TCP Tarpitting (Degradarea Vitei a Conexiunii)

- **Mecanismul de execuție:** IPS-ul permite completarea handshake-ului TCP, dar setează **TCP Window Size la 0** (Zero-Window) pe răspunsurile ulterioare.
- **Prevenirea Race Condition-ului la nivel de Kernel:** interceptarea cu NetfilterQueue/Scapy introduce un decalaj de procesare; fără o măsură de siguranță, stiva TCP/IP nativă a kernel-ului ar emite mai rapid un pachet RST, anulând capcana. Sistemul instanțiază o regulă iptables cu prioritate maximă (DROP) dedicată blocării pachetelor RST native ale kernel-ului pentru acel flux, asigurând că doar pachetul "tarpit" modificat de Scapy părăsește interfața.
- **Efectul:** fereastra zero transmite atacatorului mesajul "aștept, buffer-ul e plin", forțând scriptul de atac să mențină conexiunea deschisă și să aștepte, epuizându-i resursele.

5.3. DNS Sinkholing pentru Traficul Intern

- **Mecanismul de execuție:** dacă Pi detectează o cerere DNS către un domeniu C2 cunoscut, blochează rezoluția către serverul DNS public.
- **Delegarea Inspecției (Evitarea limitărilor eBPF):** parsarea pachetelor DNS direct în C restricționat eBPF e blocată frecvent de *eBPF Verifier* (nume de domenii cu lungime variabilă, bucle nedeterminate). Traficul DNS pe portul 53 e direcționat prin NFQUEUE către Slow-Path (Python/Scapy), unde e evaluat nativ, fără restricțiile Verifier-ului.
- **Efectul:** IPS-ul falsifică răspunsul DNS și returnează adresa stației AI. Dispozitivul infectat trimite datele exfiltrate direct către sistemul Out-of-Band, permițând analiza payload-ului și izolarea rapidă a echipamentului compromis.

Notă: acest mecanism e vulnerabil dacă dispozitivul infectat folosește DoH. Fallback-ul prin Canary Domains e documentat în 3.2.

5.4. Mecanismul de Fallback Anti-DDoS și Protecția Împotriva State Exhaustion

- **Monitorizarea Volumului (Prag PPS):** Programul eBPF menține un contor de pachete pe secundă; la depășirea pragului, aplică XDP_DROP automat.
- **Comutarea Dinamică a Verdictului:** la depășirea unui prag critic (indicând DDoS, nu exploatare manuală), IPS-ul renunță dinamic la strategiile de decepție (Tarpit/Honeypot).
- **Hard Drop (XDP_DROP):** traficul peste prag e aruncat la nivelul driverului, cu cel mai mic cost computațional posibil.
- **Concurrent Connections Limit (Protecție Slowloris / Low & Slow):** programul eBPF ține evidența stărilor deschise pe sursă într-o eBPF Map dedicată. Orice IP care menține activ un număr anormal de conexiuni simultane (peste un prag, ex: 100) primește XDP_DROP pentru pachetele noi de inițiere, salvând tabela de conntrack de la colaps.

*Politică de Evacuare pentru Contorul de Conexiuni (adăugat în v6): la fel ca harta de Dynamic Flow Offloading (3.1), acest contor per-sursă are nevoie de o politică de curățare — la XDP e greu de detectat fiabil închiderea completă a unei conexiuni (FIN/RST parțial vizibile, conexiuni terminate fără secvență completă), iar fără curățare, contorul ar putea "scurge" (crește fără a mai scădea) și bloca permanent surse legitime, în special IP-uri partajate (CGNAT, rețele de birou). Soluție: fiecare intrare din harta de conexiuni concurente poartă un timestamp al ultimei activități, verificat **inline** la fiecare pachet nou (același model $O(1)$ ca la 3.1); intrările mai vechi decât un TTL configurabil (ex: 2 minute de inactivitate) sunt tratate ca expirate și nu mai contribuie la numărătoare, fără a necesita o scanare separată a hărții.*

- **Rate Limiting Agregat (Mitigare Parțială — Atacuri Distribuite):** pragurile per-sursă nu detectează un atac distribuit (botnet) în care fiecare sursă rămâne sub prag. Programul eBPF menține, în plus, un contor global de pachete/secundă pentru întreaga interfață; la depășirea unui prag agregat, sistemul comută temporar pe un mod restrictiv (XDP_DROP pentru tot traficul neclasificat ca deja promovat). Este o plasă de siguranță, nu o soluție completă — un atac distribuit suficient de mare rămâne o limitare cunoscută a hardware-ului (vezi 7.1).

6. Implementarea Software și Stiva Tehnologică

Arhitectura software este modulară, decuplând logica de pachete (Data Plane) de logica de management și telemetrie (Control Plane).

6.1. Planul A: Implementarea Nativă eBPF/XDP

- **Componenta Kernel-Space:** un subset restricționat de **C**, compilat în bytecode eBPF via **Clang/LLVM**, verificat de *eBPF Verifier* și atașat hook-ului XDP pe eth0/eth1. Extrage headerele Layer 3/4 și aplică XDP_PASS/XDP_DROP (redirecționarea către Honeypot fiind delegată către Netfilter/conntrack, vezi 5.1).
- **Componenta User-Space: Python** (sau C/Go), folosind **libbpf/BCC**. Comunicare exclusiv prin **eBPF Maps**, inclusiv harta de "flow cache" (vezi 3.1).

6.2. Planul B: Soluția de Fallback User-Space (Netfilter Queue)

- **Mecanismul de Interceptare:** iptables/nftables pun traficul de tranzit într-o coadă dedicată spațiului utilizator.
- **Procesarea Pachetelor: Python**, folosind NetfilterQueue.
- **Inspecția și Manipularea: Scapy** pentru parsare raw, extragere SNI și manipulare headere (Tarptitting).

6.3. Interfața de Control (Control Plane API)

- **Tehnologie: FastAPI/Flask**, pe un port intern accesibil doar prin VLAN-ul de management.
- **Flux de execuție:** AI-ul trimite POST JSON (ex: {"action": "tarpit", "ip": "192.168.1.100"}); API-ul actualizează instantaneu eBPF Map (Plan A) sau regula de firewall (Plan B).

6.3.1. Autentificarea API-ului

- **Autentificare prin Token (Bearer Token):** fiecare request POST include un header `Authorization` cu un token static, stocat în variabile de mediu.

- **mTLS (recomandat):** certificate proprii semnate de un CA intern, pentru a preveni spoofing-ul de sursă și interceptarea traficului.
- **Rate Limiting intern pe API:** previne inundarea sistemului cu comenzi de blocare, chiar dacă un atacator obține token-ul.

7. Limitări Arhitecturale și Vulnerabilități Cunoscute (Known Vulnerabilities)

Acest capitol definește vulnerabilitățile cunoscute ale arhitecturii și condițiile în care sistemul poate eșua.

7.1. Vulnerabilități de tip "State Exhaustion" (Epuizarea Resurselor)

- **IP Fragmentation Attacks:** reasamblarea fragmentelor consumă memorie RAM. Mitigare — limitarea strictă a memoriei alocate și timeout agresiv (ex. 3 secunde).
- **SYN Flood:** Rate Limiting cu prag (ex. 10.000 pps) lasă o fereastră de vulnerabilitate până la atingerea pragului.
- **Fereastră de Încredere pe Fluxuri Promovate (TOCTOU):** un flux promovat beneficiază de XDP_PASS necondiționat până la TTL-ul de re-validare (3.1). Un atacator care pare benign suficient timp ar putea abuza de fereastra rămasă pentru activitate de volum mic. Mitigare — TTL scurt (3.1); risc rezidual acceptat.
- **Atacuri Distribuite (Volum Agregat Mic-per-Sursă):** un botnet cu multe surse individuale sub prag poate satura hardware-ul prin volum agregat. Mitigare parțială — contorul agregat de Rate Limiting (5.4).

7.2. Vulnerabilități de tip "Blind Spot" (Unghiuri Moarte)

- **SSL Evasion:** malware Zero-Day care imită hash-ul JA3 al unui browser legitim și comunică printr-un domeniu de încredere face IPS-ul complet orb pe Fast-Path — detecția rămâne pe seama AI-ului, post-factum.
- **ARP Spoofing (pe rețeaua internă):** un atac lansat între dispozitive din spatele Ruterului 2 nu trece prin IPS și rămâne invizibil. Mitigare externă — Dynamic ARP Inspection pe switch-ul intern.

Notă: limitările ECH și DoH sunt tratate ca blind spot-uri suplimentare, cu fallback dedicat în 3.2. Traficul IPv6 e tratat separat ca "necunoscut" în 3.4.

7.3. Vulnerabilitate de tip "Single Point of Failure" (SPOF)

Raspberry Pi-ul e singurul dispozitiv in-line pe calea de trafic; orice blocaj software sau eșec hardware întrerupe complet accesul la internet al rețelei.

- **Mitigare — Fail-Open la Nivel de Aplicație (Software Bypass):** un watchdog software independent detașează programul eBPF/XDP și comută bridge-ul pe forwarding simplu dacă serviciul de inspecție nu răspunde la heartbeat.
- **Mitigare — Hardware Watchdog:** WDT-ul integrat în SoC repornește placa automat dacă sistemul nu resetează periodic timer-ul (kernel panic, hang complet).

Combinate, cele două mecanisme acoperă atât eșecul aplicației, cât și eșecul întregului sistem de operare. Validarea e testată în scenariul 10.5.

⚠ *Limitare — Pierderea Totală a Alimentării (dacă Bypass-ul Hardware nu este achiziționat): watchdog-urile presupun ambele că placa are curent. Dacă Pi-ul pierde complet alimentarea, bridge-ul cade electric și rețeaua rămâne fără conectivitate. Soluția corectă e un Network TAP / releu de bypass fail-open la nivel electric (vezi Cap. 9), care scurtcircuitează fizic eth0↔eth1 la absența alimentării. Dacă echipa decide să nu îl achiziționeze, acest scenariu rămâne o limitare cunoscută și neacoperită.*

⚠ *Limitare — Absența Alimentării Neîntrerupte (UPS): proiectul nu include, în acest stadiu, un UPS pentru Pi, switch sau stația AI/Honeypot. O pană de curent generală ar întrerupe atât inspecția, cât și conectivitatea, indiferent de mecanismele Fail-Open. Rămâne documentată explicit ca limitare asumată, neimplementată.*

8. Fiabilitatea și Reziliența Hardware

Dat fiind rolul critic al Raspberry Pi ca unic punct de trecere a traficului, fiabilitatea stocării locale devine o cerință de arhitectură.

8.1. Stocare pe SSD prin USB 3.0

În locul cardului microSD, sistemul va rula de pe un **SSD extern conectat prin USB 3.0**, cu boot direct de pe acesta (suportat nativ de Raspberry Pi 4/5).

8.2. Read-Only Root Filesystem + tmpfs

Rootfs va fi montat **read-only** în operare normală, eliminând riscul de corupere la pierdere neașteptată de curent.

- Directoarele care necesită scriere sunt montate pe **tmpfs** (RAM disk), volatil prin natură.
- Datele care trebuie persistate sunt scrise periodic (batch) pe o partiție separată, read-write, de pe SSD.
- Beneficiu: un dispozitiv care poate fi decuplat brusc de la curent fără riscul de a nu mai porni.

8.3. Strategia de Actualizare (OTA) pe Rootfs Read-Only

- **Două partiții rootfs (A/B):** una activă (read-only, producție), cealaltă inactivă. Actualizarea se scrie pe partiția inactivă, fără a afecta sistemul care rulează.
- **Comutare la reboot + verificare de sănătate:** bootloader-ul comută pe partiția nouă; dacă verificarea eșuează, revine automat pe partiția anterioară (rollback).
- **Scop pentru stadiul de practică:** mecanismul minim care face actualizările sigure pe un rootfs imutabil, fără reflashare manuală.

8.4. Backup-ul Datelor Persistente (SSD)

- **Politică minimă de backup:** datele din partiția read-write persistentă sunt copiate periodic (ex: zilnic) către o locație externă (NAS existent sau stocare pe stația AI).

- **Limitare asumată:** nu se urmărește continuitate în timp real (RPO $\approx$ 0) — un backup zilnic e suficient pentru scopul de practică.

9. Resurse Hardware (Bill of Materials)

Lista de mai jos reflectă componentele cunoscute la momentul redactării. Cantitățile, costurile și furnizorii exacti urmează a fi completați de echipă pe măsură ce achizițiile sunt confirmate.

Componentă	Rol în arhitectură	Status / Sursă	Observații
Raspberry Pi (model TBD, recomandat Pi 4/5, min. 4 GB RAM)	Nodul 2 — IPS in-line, Fast/Slow-Path, API	De achiziționat	Modelul exact influențează fezabilitatea rulării simultane a eBPF + procesare locală
Adaptor USB 3.0-to-Gigabit Ethernet (eth1)	Interfață de ieșire a bridge-ului transparent	Oferit de mentor	Chipset necunoscut; risc — posibil fără suport XDP nativ (vezi 2.1)
SSD extern (USB 3.0)	Stocare rootfs + loguri, boot	De achiziționat	Înlocuiește cardul microSD pentru fiabilitate (vezi Cap. 8)
Ruter 1 (Gateway existent)	Nodul 1 — WAN, NAT, DHCP	Existent	—
Ruter 2 / Switch gestionabil	Nodul 3 — distribuție + posibil trunking VLAN	De verificat	Trebuie confirmat suportul 802.1Q trunking (vezi 2.2); dacă nu, se trece pe separare fizică
Stație secundară (AI + Honeypot)	Motor ML/LLM, mediu izolat Cowrie	Existentă / de confirmat	Specificații (CPU/GPU/RAM) necesare pentru Llama-3 8B / Mistral-7B local; necesită minimum o interfață fizică cu două subinterfețe 802.1Q (VLAN 99 pentru AI, VLAN 100 pentru Honeypot izolat — vezi 2.2)
Cablare Ethernet Cat5e/Cat6	Interconectare noduri	De achiziționat / existent	—
⚠ Network TAP / Releu Bypass Fail-Open (hardware)	Bypass electric eth0 $\leftrightarrow$ eth1 la	De achiziționat	Necesar pentru a acoperi complet SPOF-ul din 7.3; dacă nu se

Componentă	Rol în arhitectură	Status / Sursă	Observații
	pierdere totală de curent		achiziționează, rămâne o limitare cunoscută, neacoperită
⚠ UPS (Sursă de Alimentare Neîntreruptă)	Continuitate alimentare Pi / switch / stație AI	Neachiziționat — limitare asumată	Idee validă de robustețe suplimentară, dar rămâne în afara scopului acestei etape (vezi 7.3)

10. Metodologia de Testare, Scenarii de Atac și Criterii de Validare

Sistemul va fi supus unei serii de teste de penetrare și de stres într-un mediu de laborator izolat, folosind o stație dedicată de atac (ex: Kali Linux) și generatoare de trafic sintetic.

10.1. Scenariul 1: Atac Volumetric (Testarea supapei Fast-Path)

- **Vector de Atac:** SYN Flood direcționat către un serviciu intern.
- **Unelte:** hping3 sau scripturi customizate (>20.000 pachete/secundă).

Rezultatul Așteptat: pragul de Rate Limiting e identificat în < 1 secundă; mecanismele de decepție sunt ocolite; sistemul comută pe XDP_DROP.

- **Metrica de succes:** CPU pe Raspberry Pi nu depășește 50-60%; traficul legitim continuă cu latență adăugată minimă.

10.2. Scenariul 2: Exfiltrare Mascată (Testarea Slow-Path și ML)

- **Vector de Atac:** trafic HTTPS/TLS complet criptat, folosit pentru beacon-uri C2 și exfiltrare pe port non-standard.
- **Unelte:** payload-uri Meterpreter mascate sub SSL, sau script Python personalizat.

Rezultatul Așteptat: Slow-Path extrage metadatele; Isolation Forest detectează deviația; AI-ul trimite comanda de blocare prin API autentificat (6.3.1).

- **Metrica de succes:** bucla de feedback se închide în < 5 secunde; conexiunea e terminată.

10.3. Scenariul 3: Exploatare Interactivă (Testarea Honeypot și LLM XAI)

- **Vector de Atac:** Brute Force pe portul 22, urmat de autentificare și comenzi Post-Exploitation.
- **Unelte:** Hydra, logare manuală.

Rezultatul Așteptat: traficul e preluat de Netfilter/conntrack, DNAT către Honeypot (conform 5.1); logurile Honeypot conțin direct IP-ul real al atacatorului (fără corelare suplimentară, vezi 5.1/4.2).

- **Metrica de succes:** LLM-ul generează un raport JSON XAI corect, încadrând tehnicile în MITRE ATT&CK, cu halucinații zero.

10.4. Indicatori Cheie de Performanță (KPIs) ai Rețelei

- **Throughput:** degradarea vitezei (iperf3) pentru traficul curat nu trebuie să depășească 10-15% din capacitatea gigabit nativă.
- **Latență Adăugată (Jitter):** extragerea de metadate în Slow-Path adaugă cel mult 2-5 ms pentru pachetele de handshake.
- **False Positive Rate (FPR):** rata de blocare accidentală a traficului normal sub 1% pe 24 de ore de test.

Recalibrare KPI (v6): întrucât 5.1 impune activarea globală a br_netfilter, iar 3.1.1 introduce optimizarea NOTRACK pentru traficul deja promovat, KPI-ul de Throughput de mai sus trebuie măsurat explicit în ambele condiții — cu și fără NOTRACK — pentru a documenta cât de mult din costul br_netfilter e efectiv absorbit de optimizare. Dacă degradarea rămâne peste 10-15% chiar și cu NOTRACK aplicat, pragul trebuie renegociat cu mentorul, nu forțat artificial.

10.5. Scenariul 4: Test de Failover (Validarea Mecanismelor Fail-Open)

- **Vector de Test:** blocaj software (terminare forțată a procesului de inspecție) și, separat, kernel panic simulat în laborator.

Rezultatul Așteptat: watchdog-ul software comută bridge-ul pe forwarding simplu în cazul blocajului; watchdog-ul hardware repornește placa în < 30 secunde la kernel panic.

- **Metrica de succes:** zero minute de indisponibilitate totală pentru scenariul software; timp de recuperare sub pragul definit pentru kernel panic.